

 INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO	INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO CAMPUS – São Paulo	
Disciplina: SPOWEB1 / AVDI		
Desenv. Web 1 / Artes Visuais	Projeto integrador—Interface—HTML- CSS	Data.: 04/05/2026
Nome do Aluno: Matheus Roberto Santos Pereira Prontuário: Sp3193641		
Professora: Claudete de Oliveira		
Link do Github: https://github.com/Matheus-Robert/Projeto-integrador---Interface-HTML-CSS		

Primeiro arquivo: Index.html

Enunciado:

Criar uma página com um campo para inserir um número, um botão para validar e um espaço para exibir mensagens, utilizando CSS e JavaScript externos.

Raciocínio Passo a passo:

O código começa definindo a estrutura básica da página e preparando o ambiente para exibir o conteúdo. Em seguida, organiza os elementos dentro de um espaço centralizado (container), onde é apresentado um título para indicar a finalidade da página. Logo depois, é criado um campo para que o usuário insira um número e um botão que permite iniciar a validação desse valor. Abaixo desses elementos, há um espaço reservado para exibir mensagens ao usuário, que serão mostradas conforme a interação. Por fim, o código garante a conexão com os arquivos externos responsáveis pela aparência e pelo funcionamento da página, permitindo que tudo opere corretamente.

Código do arquivo index.html:

```
<!DOCTYPE html>

<!-- Define que o documento é HTML5 -->

<html lang="en">

<!-- Define o idioma da página em inglês -->

<head >

    <!-- Cabeçalho do documento (configurações) -->
```

```

<meta charset="UTF-8">

<!-- Define o conjunto de caracteres (acentuação correta) -->

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Faz o site se adaptar a diferentes tamanhos de tela -->

<title>Exemplo de Desvio Condicional - Validação Número</title>

<!-- Título da aba do navegador -->

<link rel="stylesheet" href="style.css">

<!-- Importa o arquivo de estilos CSS -->
</head>

<body>

<!-- Corpo da página (tudo que aparece para o usuário) -->

Escreva as instruções aqui

<!-- Texto exibido diretamente na página -->

<div class="container">

<!-- Container principal para organizar o conteúdo -->

<h1>Validação de números </h1>

```

```

    <!-- Título principal da página -->

    <input type="number" id="numero" name="numero" placeholder="Digite um
número">

    <!-- Campo de entrada numérico -->

    <!-- id="numero" permite acessar esse campo no JavaScript -->

    <!-- placeholder mostra um texto dentro do input -->

    <button onclick="validarNumero()">Validar</button>

    <!-- Botão que executa a função validarNumero() ao ser clicado -->

    <p id="mensagem"></p>

    <!-- Parágrafo onde será exibida a mensagem de resultado -->

    <!-- id="mensagem" permite manipular via JavaScript -->

</div>

<script src="script.js"></script>

<!-- Importa o arquivo JavaScript com a lógica da validação -->

</body>

</html>

```

Explicação Técnica:

O código HTML inicia com a declaração `<!DOCTYPE html>`, que define o documento como HTML5. Em seguida, a tag `<html>` envolve todo o conteúdo da página e possui o atributo `lang`, que indica o idioma. Dentro dela, a seção `<head>` contém metadados como a codificação de caracteres (`<meta charset="UTF-8">`), configuração de responsividade (`<meta name="viewport">`), o título da página (`<title>`) e o vínculo com o arquivo de estilos externo por meio da tag `<link>` com `rel="stylesheet"` e `href="style.css"`.

No `<body>`, onde fica o conteúdo visível, há uma `<div>` com a classe "container" que organiza os elementos da interface. Dentro dessa div, a tag `<h1>` define o título principal da página. O campo `<input>` com `type="number"` permite a entrada de dados numéricos e utiliza os atributos `id` e `name` para identificação, além do placeholder que exibe uma mensagem orientativa ao usuário.

O botão `<button>` possui o atributo `onclick`, que chama a função `validarNumero()` quando acionado, estabelecendo a interação com o JavaScript. A tag `<p>` com `id="mensagem"` funciona como área de saída para exibir mensagens dinâmicas ao usuário.

Por fim, antes do fechamento da tag `</body>`, é inserida a tag `<script>` com o atributo `src="script.js"`, responsável por importar o arquivo JavaScript que contém a lógica da aplicação.

Segundo arquivo: style.css

Enunciado:

Criar uma folha de estilo para estilizar a página, centralizando o conteúdo na tela, definindo fonte, cores, espaçamentos e aparência dos elementos como container, campo de entrada, botão e mensagem, proporcionando uma melhor experiência visual ao usuário.

Código do arquivo style.css:

```
body{
  font-family: Arial, sans-serif;
  /* Define a fonte do texto */

  display: flex;
  /* Ativa o flexbox para organização dos elementos */

  justify-content: center;
  /* Centraliza horizontalmente */

  align-items: center;
  /* Centraliza verticalmente */

  height: 100vh;
  /* Faz o body ocupar 100% da altura da tela */

  background-color: #f0f0f0;
  /* Define a cor de fundo da página */
}
```

```

}

.container {
  /* Classe do container (OBS: nome está escrito como "cotainer") */

  text-align: center;
  /* Centraliza o texto dentro do container */

  padding: 20px;
  /* Espaçamento interno */

  background-color: #fff;
  /* Fundo branco */

  border-radius: 10px;
  /* Bordas arredondadas */

  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  /* Sombra leve ao redor do container */
}

input{
  padding: 10px;
  /* Espaçamento interno do campo */

  margin-right: 10px;
  /* Espaço à direita do input */

  border: 1px solid #ccc;
  /* Borda cinza clara */

  border-radius: 5px;
  /* Bordas arredondadas */
}

button {
  padding: 10px 20px;
  /* Espaçamento interno do botão */

  background-color: #007Bff;
  /* Cor de fundo azul */

  color: white;
  /* Cor do texto */

  border: none;
  /* Remove a borda padrão */
}

```

```

border-radius: 5px;
/* Bordas arredondadas */

cursor: pointer;
/* Muda o cursor para "mãozinha" ao passar em cima */
}

button:hover {
background-color: #0056b3;
/* Cor do botão ao passar o mouse */
}

p{
margin-top: 20px;
/* Espaço acima do parágrafo */

font-size: 18px;
/* Tamanho da fonte */
}

```

Explicação Técnica:

O arquivo CSS define a aparência visual da página. A regra aplicada ao body utiliza display: flex junto com justify-content: center e align-items: center para centralizar o conteúdo horizontal e verticalmente na tela, enquanto height: 100vh garante que o corpo ocupe toda a altura da janela. A propriedade font-family define a fonte padrão e background-color aplica uma cor de fundo.

A classe .container (responsável pelo bloco principal) aplica text-align: center para centralizar o conteúdo interno, padding para espaçamento interno, background-color para o fundo branco, border-radius para bordas arredondadas e box-shadow para adicionar uma sombra ao redor do elemento.

O seletor input estiliza o campo de entrada com padding, margin-right, border e border-radius, melhorando a usabilidade e aparência. O button recebe propriedades como padding, background-color, color, border, border-radius e cursor, enquanto o pseudo-seletor button:hover altera a cor de fundo ao passar o mouse, criando um efeito interativo.

Por fim, o seletor p define o espaçamento superior (margin-top) e o tamanho da fonte (font-size) para a área onde as mensagens são exibidas.

Terceiro arquivo: script.js

Enunciado:

Criar uma função que valide o número digitado, exibindo mensagens diferentes caso o campo esteja vazio, seja maior que 10, maior que 5 ou menor ou igual a 5, com cores diferentes.

Código do arquivo script.js:

```
function validarNumero(){
    // Função chamada ao clicar no botão

    let numero= document.getElementById("numero").value;
    // Pega o valor digitado no input com id "numero"

    let mensagem = document.getElementById("mensagem");
    // Pega o elemento <p> onde a mensagem será exibida

    if (numero=="") {
        // Verifica se o campo está vazio

        mensagem.textContent= "Por favor insira um número.";
        // Define o texto da mensagem

        mensagem.style.color="red";
        // Define a cor da mensagem como vermelho
    }
    else{
        // Caso o campo não esteja vazio

        numero=parseInt(numero);
        // Converte o valor digitado (texto) para número inteiro

        if(numero>10){
            // Verifica se o número é maior que 10

            mensagem.textContent="O número é maior que 10.";
            // Exibe a mensagem correspondente

            mensagem.style.color="green";
            // Define a cor como verde

        } else{
            // Caso não seja maior que 10

            if(numero>5){
                // Verifica se o número é maior que 5

                mensagem.textContent="O número é maior que 5, mas menor ou igual a
10.";
                // Exibe a mensagem correspondente

                mensagem.style.color="orange";
                // Define a cor como Laranja

            } else{
```

```

        // Caso o número seja 5 ou menor

        mensagem.textContent="O número é 5 ou menor.";
        // Exibe a mensagem correspondente

        mensagem.style.color="blue";
        // Define a cor como azul
    }

}

}

}

```

Explicação Técnica:

O arquivo JavaScript define a função `validarNumero()`, responsável pela lógica da aplicação. Inicialmente, é utilizado `document.getElementById()` para acessar o valor do campo de entrada (`numero`) e o elemento de saída (`mensagem`). O valor do input é obtido pela propriedade `.value`.

A estrutura condicional começa verificando se o campo está vazio (`numero == ""`). Caso verdadeiro, o conteúdo do elemento `mensagem` é atualizado por meio da propriedade `textContent`, e sua cor é definida com `style.color`.

Se o campo não estiver vazio, o valor é convertido de texto para número inteiro utilizando `parseInt()`. Em seguida, são realizadas verificações condicionais: se o número for maior que 10, uma mensagem específica é exibida com cor verde; caso contrário, outra verificação analisa se o número é maior que 5, exibindo a mensagem correspondente; se nenhuma das condições anteriores for atendida, o sistema considera que o número é menor ou igual a 5, exibindo a mensagem adequada com cor azul.

Essas estruturas condicionais (`if` e `else`) controlam o fluxo da aplicação, permitindo respostas diferentes conforme o valor inserido pelo usuário.

Conclusão:

Este trabalho permitiu compreender, na prática, a integração entre HTML, CSS e JavaScript no desenvolvimento de páginas web interativas. Por meio da criação de uma interface simples, foi possível estruturar elementos com HTML, aplicar estilos visuais com CSS e implementar lógica de validação utilizando JavaScript.

A atividade demonstrou a importância da organização do código e da utilização correta de cada tecnologia, destacando como pequenas interações, como a validação de um campo,

podem melhorar a experiência do usuário. Além disso, reforçou o entendimento sobre estruturas condicionais e manipulação de elementos da página em tempo real.

Dessa forma, conclui-se que a combinação dessas três linguagens é essencial para o desenvolvimento web moderno, permitindo a criação de aplicações funcionais, dinâmicas e visualmente agradáveis.

Referências

OPENAI. *ChatGPT: ferramenta de inteligência artificial para suporte ao desenvolvimento web*. 2026. Disponível em: <https://chat.openai.com/>. Acesso em: 4 maio 2026.